

Lab 2 – Hello Hadoop!

CC66I-1 – July 21, 2026

En esta sesión implementaremos un contador de palabras simple y lo ejecutaremos usando Hadoop (una implementación open-source de MapReduce). El principal objetivo de este laboratorio es que aprendas a interactuar con HDFS (Hadoop Distributed File System), subir/cargar datos, implementar una tarea básica de Hadoop en Java, lanzarla en el cluster y obtener resultados.

- Primero queremos conectarnos al servidor vía SSH.
 - En Windows con PuTTY:
 - * Dirígete a <http://aidanhogan.com/teaching/tools/cc5212/putty.exe> y <http://aidanhogan.com/teaching/tools/cc5212/winscp553.zip> para descargar las herramientas. Estas son herramientas exclusivas para Windows para poder usar SSH y SFTP/SCP.
 - * Abre PuTTY. En host-name escribe `cm.dcc.uchile.cl` puerto 220. Haz clic en Save. Haz clic en Open.
 - * Ingresar y
 - * La es `uhadoop` y la se le proporcionará durante la clase.
 - En un terminal (Linux/Mac/Windows):
 - * `ssh -l  cm.dcc.uchile.cl -p 220`
 - * Ingresar y
 - * La es `uhadoop` y la se le proporcionará durante la clase.
 - Ahora estás en el servidor maestro de un cluster con cuatro máquinas
 - Dado que todos están usando un solo username, por favor, *por favor* sé considerado. **Piénsalo dos veces antes de utilizar comandos como `rm -r`, `kill` y similares.**
- Ahora queremos navegar por el servidor de archivos distribuido y cargar algunos datos:
 - Escribe `hdfs dfsadmin -report` para ver el estado del HDFS.¹
 - Escribe `hdfs dfs`.² Esto mostrará las opciones que existen para interactuar con HDFS.
 - Escribe `hdfs dfs -ls /` para ver los contenidos en el directorio raíz (root). Estos archivos están almacenados en todos los servidores de manera distribuida.
 - Escribe `hdfs dfs -ls /uhadoop2026/cc66i` para ver su contenido. Luego inventa un nombre de usuario para tu grupo. Escribe: `hdfs dfs -mkdir /uhadoop2026/cc66i/` reemplazando con el nombre que inventaste, p.e., `hdfs dfs -mkdir /uhadoop2026/cc66i/amoya`. De ahora en adelante esta será tu carpeta personal... por favor mantén tus archivos del DFS (sistema de archivos distribuido) en esta carpeta. También ten presente que cualquiera puede accederla, así que no subas datos privados.
 - Ahora en el sistema de archivos *local*, dirígete a la carpeta `cd /data/2026/uhadoop/cc66i/`. Crea un directorio propio `mkdir` con el mismo nombre de usuario anterior.
 - Mira en la carpeta `ls -l /data/uhadoop/shared/wiki` del sistema de archivos local del maestro. Aquí encontrarás almacenado en el disco local el archivo sobre el cual queremos contar palabras. Necesitas cargarlo al DFS. Escribe (sin salto de línea):
 - * `hdfs dfs -copyFromLocal /data/uhadoop/shared/wiki/es-wiki-abstracts.txt.gz /uhadoop2026/cc66i//es-wiki-abstracts.txt.gz`
 - Verifica que ahora esté ahí: escribe `hdfs dfs -ls /uhadoop2026/cc66i/`.

¹Ignora advertencias así `WARNING: ....` No importan.

²También se puede escribir `hadoop fs` pero al parecer se prefiere más `hdfs dfs`.

- Ahora que has cargado los datos al DFS, necesitamos ejecutar la tarea Hadoop para contar palabras.
 - Descarga el proyecto desde Material Docente y ábrelo en Eclipse.
 - Abre la clase `WordCount`. Aquí tenemos la clase mapeadora, la clase reductora y el main.
 - Para empezar, revisa brevemente la clase `WordCountMapper`. Esta clase decide cómo dividir los datos y distribuir los “trozos” entre las máquinas.
 - No necesitamos hacer nada especial con las comparaciones, ordenamientos, particionado, etc. Todas las configuraciones por defecto son suficientes para nosotros, así que podemos saltarnos esta parte y seguir con...
 - ... el `WordCountReducer` que suma los conteos emitidos por la clase mapeadora.
 - Finalmente tenemos el main que define la configuración de la tarea y el control de flujo. En este caso, ojo que podemos usar `WordCountReducer` como un combiner también porque suma valores.
- Necesitamos compilar y ejecutar el código.
 - En el proyecto en Eclipse, haz clic derecho en `build.xml`, luego en `Run As ...` y haz clic en `Ant Build`. Si falla, podrías necesitar crear manualmente una nueva carpeta `dist` en la carpeta principal del proyecto. Actualiza el proyecto (F5) y chequea que se genere el archivo JAR.
 - * Si recibes un error que diga algo sobre `javac1.8` o similar, haz clic derecho en `build.xml`, anda a `External Tools Configurations` y agrega la línea `-Dbuild.compiler=javac1.7` como argumento.
 - Necesitas copiar el archivo JAR al servidor.
 - * En Windows con WinSCP: Abre WinSCP. Selecciona SCP. En hostname pon `cm.dcc.uchile.cl` y el puerto 220. Escribe `uhadoop` como username. No escribas la password todavía. Haz clic en Save para guardar. Cuando se solicite, ingresa la misma contraseña que usaste para PuTTY. Copia tu archivo `.jar` en `/data/2026/uhadoop/cc66i/[USUARIO]`.
 - * En un terminal de Linux/Mac/Windows (todo un solo comando) desde la carpeta `dist/` de tu proyecto con el archivo `.jar` en tu máquina:


```
scp -P 220 gdd-hadoop.jar [CUESTA]@cm.dcc.uchile.cl:/data/2026/uhadoop/cc66i/[USUARIO]/3
```
 - Ahora solo necesitas hacer la llamada a la tarea Hadoop. Devuélvete a la sesión de SSH. Ejecuta (todo en un solo comando):


```
* hadoop jar /data/2026/uhadoop/cc66i/[USUARIO]/gdd-hadoop.jar WordCount
   /uhadoop2026/cc66i/[USUARIO]/es-wiki-abstracts.txt.gz
   /uhadoop2026/cc66i/[USUARIO]/wc/
```

El directorio de salida *no* debe existir previo a la ejecución de la tarea!

Deberías poder ver el progreso de Map/Reduce a medida que se ejecuta. Asimismo, deberías poder ver los resultados del contador. En el registro, deberías ver una línea que dice:

```
INFO mapreduce.Job: Running job: job_XXXXXXXXXXXX-YYYY
```

donde `job_XXXXXXXXXXXX-YYYY` es el id de tu proceso. Vale anotar ese id para luego poder revisar en qué estado está. Muchas veces un cluster de MapReduce/Hadoop va a ser compartido con mucha gente, y puede haber colas de procesos esperando. Si quieres ver la cola, puedes poner

```
* mapred job -list
```

Si quieres ver el estado de un proceso con ID `job_XXXXXXXXXXXX-YYYY`:

```
* mapred job -status job_XXXXXXXXXXXX-YYYY
```
 - Cuando finalice, será el momento de inspeccionar los resultados (escritos en HDFS).
 - * Ejecuta `hdfs dfs -ls /uhadoop2026/cc66i/[USUARIO]/wc/` para ver el archivo de salida.⁴
 - * Ejecuta `hdfs dfs -cat /uhadoop2026/cc66i/[USUARIO]/wc/part-r-00000 | more` para inspeccionar el archivo principal de salida (puede haber varias partes asociadas con diferentes máquinas pero acá habrá una). Presionando `Espacio` puedes ver más del archivo y presionando `Q` (Quit) puedes terminar de revisar el archivo.

³Ojo que el port en SSH se da con `-p` pero en SCP con `-P`.

⁴Esta vez hay sola una reducción con un archivo de salida. Podría haber más archivos: en general, un archivo por reductor.

- * Intenta encontrar la cuenta para la palabra “de” ejecuta:
 - `hdfs dfs -cat /uhadoop2026/cc66i/[USUARIO]/wc/part-r-00000 | grep -P "^de\t"`.
 ¿Obtuviste (algo similar a) 4.921.139?
- Ahora vas a crear tu propia tarea para contar el número de palabras únicas que usen cada letra. Es decir, para cada letra, queremos encontrar el número de palabras únicas que usen la letra.
- Por ejemplo, para las palabras `casa`, `puerta` y `cama`, la salida debiera ser:

a	3
c	2
e	1
m	1
...	
- * Copia `WordCount.java` con el nombre `LetterCount.java`. Además renombrar `WordCountMapper` y `WordCountReducer` adentro del archivo a `LetterCountMapper` y `LetterCountReducer`. Ahora modificarás `LetterCount.java` para hacer lo necesario. Algunas indicaciones:
 - Puedes considerar la salida de `WordCount` como entrada. Cada par de la entrada tendrá el formato (l, v) donde l no importa (da la posición actual en bytes del archivo) y v es un `Text` de la forma “palabra n” (p.ej., “de 4921139”) separado por tab (`\t`). La entrada ya está en minúscula.
 - Incluiremos letras alfabéticas (en cualquier alfabeto); es decir, incluiremos los caracteres `c` tal que `Character.isAlphabetic(c)` en Java es verdadero.
 - Se consideran las letras con diacríticos (á, ñ, etc.) como distintas letras.
 - Puedes usar `Text` (o `IntWritable` con casting) para representar cada letra en Hadoop.
 - Si una palabra tiene la misma letra varias veces, cuenta una sola vez.
 - Cuando esté listo, hay que recompilar y copiar el JAR de nuevo.
 - El comando será:


```
hadoop jar /data/2026/uhadoop/cc66i/[USUARIO]/gdd-hadoop.jar LetterCount
/uhadoop2026/cc66i/[USUARIO]/wc/ /uhadoop2026/cc66i/[USUARIO]/lc/.
```

 No hay que dar el nombre del archivo de entrada. Si la carpeta `.../wc/` tuviera varios archivos de salida de una tarea previa, Hadoop incluiría todos como entrada (en paralelo).
- OPCIONAL: ahora intenta implementar una segunda tarea MapReduce para ordenar las palabras y/o las letras por ocurrencia.
- ENTREGA por U-Cursos el `LetterCount.java` y los primeros 26 resultados para `a-z`.